A Novel Real-time Anti-spam Framework

DI WU, WEI SHI, and XIANGYU MA, Carleton University, Canada

As one of the most pervasive current modes of communication, email needs to be fast and reliable. However, spammers and attackers use it as a primary channel to conduct illegal activities. Although many approaches have been developed and evaluated for spam detection, they do not provide sufficient accuracy. This deficiency results in significant economic losses for organizations. In this article, we first propose a framework for creating novel spam filters using Keras to combine a Convolutional Neural Network (CNN) with Long Short-Term Memory (LSTM) classification models. We then use this framework to introduce a specific solution applicable to realistic scenarios involving dynamic incoming email data in real-time. This solution takes the form of a real-time content-based spam classifier. We evaluate its performance concerning accuracy, precision, recall, false-positive, and false-negative rates. Our experimental results show that our approach can significantly outperform existing solutions for real-time spam detection.

CCS Concepts: • **Information systems → Clustering and classification**; • **Security and privacy → Artificial immune systems;**

Additional Key Words and Phrases: Spam filtering, cybersecurity, cybercrime, spam intrusion detection, content based filters, neural networks, machine learning, deep learning, natural language processing, text representation, CNN, RNN, LSTM

ACM Reference format:
Di Wu, Wei Shi, and Xiangyu Ma. 2021. A Novel Real-time Anti-spam Framework. *ACM Trans. Internet Technol.* 21, 4, Article 88 (September 2021), 27 pages.
https://doi.org/10.1145/3423153

1 INTRODUCTION

Email has been used as one of the main interpersonal communication tools for decades. The usage of email continues to grow each year [18]. It is used not only for basic interpersonal communication but also for many official intra-organizational or cross-organizational communication purposes such as ordering, billing, and managing contracts and announcements. Email communication is commonly embedded into business work flow processes in many organizations. This has caused email to become one of the primary channels used by spammers and attackers to conduct illegal campaigns on the Internet. The term "spam" is defined as unsolicited commercial email that is sent indiscriminately [14] and "ham" is defined as email that is not spam [32]. The definition implies three main characteristics: (1) spam is not requested by a recipient; (2) spam is commercially driven;

Authors' addresses: D. Wu, Carleton Univerisity, 1125 Colonel By Dr. K1S 5B6, Ottawa, Canada; email: di.wu@carleton.ca; W. Shi and X. Ma, Carleton University, 1125 Colonel By Dr. K1S 5B6, Ottawa, Canada; emails: wei.shi@carleton.ca, johnnyma@cmail.carleton.ca.
Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2021 Association for Computing Machinery.
1533-5399/2021/09-ART88 $15.00
https://doi.org/10.1145/3423153

(3) spam is sent in bulk. Most email users do not know the implication of spam, because it does not look very dangerous to them. In fact, spam has a significant negative impact on businesses and organizations, as it not only wastes resources and time but also puts users and organizations under security threats. For example, spam can contain a link to phishing sites, on which a single click can cause serious virus infections [4]. According to the Symantec Intelligence Report in 2018 [10], the proportion of spam over all incoming emails was 55% in 2018. Enterprises tend to be more affected by spam attacks, since, for most of them, email is their primary method of communication [10]. In 2018, the top ransom-ware distribution method was deployed through spam [10].

Consequently, implementing an efficient and accurate spam filter is necessary for all levels of usage of emails: individual recipients, groups of recipients, organizations, and cloud email clients. Several spam classification algorithms have been developed and implemented to detect spam. However, achieving high accuracy in the detection of spam constitutes a great challenge. Spam filters can be divided into two main categories: non-content-based and content-based information [29]. A novel spam filter framework is proposed in this research including both categories as its two main components. Non-content information mainly refers to email headers and email related traffic. Email headers include the routing information of the email and related sender and recipient information, such as the sender's IP address, reputation, time stamps, and so on. Knowledge engineering is the main strategy applied to tackle non-content information spam checks. In our framework, we propose the use of the following rule-based techniques as the non-content-based spam filter component: signature-based techniques, DNS-based techniques, and behavior-anomaly-based techniques. A content-based spam filter refers to the use of text mining techniques for mining information contained in email subjects and bodies. Combining spam identification algorithms with data pre-processing techniques and classification models is challenging due to the nature of mining the unstructured data containing meaningless text in a multitude of text files. In our proposed content-based spam filter component, we first pre-process the data and extract features from the textual information contained in the email. Then a feedback learning process is engaged in our proposed solution to interact with **Machine Learning (ML)** and **Deep Learning (DL)** models to improve the accuracy and efficiency of the spam identification process.

Handling "Zero-day" spam is a challenging task. Anti-spam strategies, like most anti-virus solutions, need to be updated dynamically. The race between spammers and spam-filtering methods never ends. A spam filter considered the best at one point in time may be out of date within a week or less. Spammers always try to use hosts that are not excluded yet or use newly developed spam, referred to as "Newborn Spam" or "Zero-hour Spam" or "Zero-day Spam," to avoid known methods/algorithms in spam detection. Most of the current ML spam detection techniques work only on static sets of collected email content. In other words, these techniques do not update their knowledge base in real-time. The use of real-time updates is essential for the precise classification of spam that is hidden in the massive non-stop flow of inbound emails. In this article, we first analyze the efficiency of existing email filter solutions and propose our framework, which is designed with a layered real-time solution. In this solution, we use: (1) a daily updated hybrid DL/ML intelligent content-based spam engine as the top layer, (2) daily updated spam and malware signatures, and (3) a rule-based non-content-based filter component as the bottom layer to detect and prevent spam more efficiently and dynamically.

The rest of the article is organized as follows: We conduct an in-depth theoretical analysis and a literature review on the common DL/ML methods used as spam filters, including **Naïve Bayes (NB), Support Vector Machines (SVM), Convolutional Neural Networks (CNN), K-Nearest Neighbor (KNN)**, and **Long Short-Term Memory (LSTM)** in Section 2. We also summarize various ML algorithms and analyze their strengths and weaknesses. In Section 3, we describe existing research challenges, which lead to the problem statement. In Section 4, we explain the

conceptual structure of the proposed novel anti-spam framework architecture and provide a detailed description of the content-based spam filter component. In Section 5, we present the experiments and evaluations conducted. We compare our experimental results against existing methods using a popular spam dataset and provide a detailed analysis after. We make conclusions and outline future work in Section 6.

2 RELATED WORK

2.1 Datasets

There are five widely used datasets for the proposed research: Spambase, SpamAssassin, Enron Spam Corpus, Ling Spam, and CSDMC2010 Spambase.

Although much of the research on spam filters has used the Spambase dataset for experiments, this dataset has some limitations. The spam is selected from one mailbox at one company instead of a general spam sample. In Reference [23], it is shown that the dataset is not suitable for developing a general spam filter solution by comparing it to another dataset, SpamAssassin, which is a bigger and more general dataset [23]. SpamAssassin is the only one among the five datasets that has email header information. As with SpamAssassin, the CSDMC2010 dataset has both hams and spam that are collected randomly instead of from one individual mailbox. Both CSDMC2010 Spambase and Enron Spam Corpus include variants of spam words, which means they treat all morphological variants of a common root word in the same way [12]. Ling Spam is the smallest of the five datasets. The spam rate of this dataset is also smaller than the others. The special aspects of the Ling Spam dataset are the focus on linguistic interests around job postings, research opportunities, and software discussion. This dataset consists of randomly collected discussions from a linguistics forum rather than from specific mailbox.

2.2 Non Machine Learning-based Anti-spam Solutions

Many anti-virus researchers spend an enormous amount of time and effort examining samples of code from malware and applying a variety of techniques to identify practical signatures by using auto-generated signatures [13]. Signature-based spam filtering is a technique that uses malware executable signatures and/or rules to spot spam. The signatures are extracted from known spam content [13]. There are generic signatures and auto-generated signatures. A DNS-based spam filter is a technique to detect spammers' DNS traffic by using DNS logging to spot suspected IP addresses. However, because spammers usually try to use a peer-to-peer structure, the IP address is spread out and there is no central control node server. This leads to a barrier in detecting bots, also called the infected hosts [5]. Behavior anomaly-based spam filtering is a technique to monitor suspicious email traffic. When mismatched behavior is found by comparing it to predictable and normal behavior, it will be closely observed to determine if it is spam [39].

2.3 Machine Learning-based Anti-spam Solutions

Having a spam filter that works for everyone is very difficult, although spam filtering research has been an active field for many years and will continue to improve. Two-step data analysis models, involving data processing followed by classification, can serve populations of users on individual, group, corporation, and cloud customer levels [22]. Hackers are very sophisticated at designing phishing scams that appear to be legitimate within the domain of an organization or community but are attempting to gather confidential information or infect a computer network with viruses.

2.3.1 Data Pre-processing and Feature Extraction. To improve the accuracy of spam classification, it is important to validate the quality of the input data. The data collected from the emails cannot be directly used for classification, because it usually contains noisy or incorrect values.

Data pre-processing is the first main step before doing any mining. Data pre-processing includes noise removal, tokenization, stemming, and lemmatization [30].

For the feature extraction, a corpus of annotated emails is automatically analyzed using text mining algorithms to extract useful information such as words and various text statistics, enabling spam detection [31]. The **Bag-of-Words (BoW)** model with an unstructured set of word tokens is usually used to classify between spam and legitimate messages [35]. However, spam content can vary dramatically between different domains targeted by spammers. For instance, in the context of a university email system, spam content is mainly related to requesting one's confidential information for program or course registration, scholarships, conferences or enrollment, whereas in a health organization email system, spammers may ask one's confidential information for donating research funding towards a non-existing medication or paying for a new therapy [31]. Domain-specific spam features can be more efficient than general-purpose ones for spam filtering [31]. Having richer semantic support to distinguish between emails containing identical sets of words but different semantic structures is one way of achieving this goal. Recent research has shown several other spam detection models that outperform BoW. These include the enhanced **Topic-based Vector Space Model (eTVSM)**, several **Latent Semantic Indexing (LSI)** models, and the **Latent Dirichlet Allocation (LDA)** model [31].

2.3.2 Classifiers. ML approaches have been used in classification models for spam detection research for nearly two decades and most of them achieve very good accuracy and performance measurements on string matching algorithms [9, 12, 16]. To encode text into computer-readable mathematical features, an existing work [12] compared various well-known string-matching ML methods on spam classification, including **Longest Common Subsequence (LCS)**, Jaro, Jaro-Winkler, **Levenshtein Distance (LD)**, Bigram and **Term Frequency-Inverse Document Frequency (TFIDF)**. Experiments were done on two spam datasets: Enron Spam Corpus and CSDMC2010, with a list of spam keywords. The experimental results reveal that Bigram achieves the best performance with over 99.8% accuracy, over 99.9% precision, and almost 100% recall on all thresholds. In Reference [16], four ML classification techniques are applied to the UCI ML repository 1999 spam dataset in the WEKA environment. They are BayesNet, J48 [8], Naïve Bayes, and Lazy-IBK. Experimental results show that J48, an ML classifier using a **Decision Tree (DT)**, is the best algorithm with an accuracy of 85.06%. It also outperforms others in the recall, precision, **False Positive Rate (FPR), False Negative Rate (FNR)**, and F-measure measurements. In Reference [34], the researchers review various ML classification algorithms and discover **Rotation Forest (RF)** to have the best performance, with an accuracy of 94.2%, while other common algorithms such as J48 DT, Naïve Bayes, and Multilayer Perceptron have accuracies of 92.3%, 88.5%, and 93.2%, respectively. In Reference [12], the researchers also propose a hybrid method to classify spam by using K-means clustering and SVM. They find that the processing time of the task and FPR decrease dramatically and the detection rate is improved compared to the SVM-only based spam detection. The accuracy of classification for this approach is 98.01% and the FPR is 0.04% [12].

2.4 Deep Learning-based Anti-spam Solutions

To avoid the manual feature engineering process, one way is to use DL neural networks by identifying the hidden features from the data with "self-learned" extraction. DL is a subset of ML consisting mainly of neural networks with more complex layers for automatic pattern classification as well as self-learned feature learning [19, 24]. However, the traditional approach to deal with textual data has always used a vector space model, which encodes each word separately. It is therefore difficult for neural networks to process the data. A method called word embedding is applied to address this deficiency by translating semantic textual information into continuous dense features to learn

the distributed representation of words [6, 7, 21]. Research studies have shown that DL completely surpasses traditional ML methods [3, 7, 21, 25, 36]. In Reference [37], a neural network with two hidden layers was constructed, using 30 nodes in each hidden layer. Training is set up using 2,000 batches where each batch consists of three data entries and uses SpamBase as the dataset. The experiment results show the neural network achieved higher accuracy than NB in email classification. In Reference [3], the backpropagation algorithm model is tested, and the highest overall accuracy is around 85%. CNN is one of the popular DL models that can extract relevant features from the data. The CNN model involves a deeper structure, as the layers are sparsely connected rather than fully connected. In Reference [27], a CNN-based SMS spam filter is proposed that achieves an accuracy of 98.4%. In References [19, 20], the authors compare **Gated Recurrent Neural Networks (GRNN)** LSTM with CNN and **Recurrent Neural Network (RNN)**. They show that gates allow the network to accumulate information over a long duration. They further investigate LSTM-based algorithms for anomaly detection and report significant performance improvements over conventional DL methods. Various ML-based algorithms and DL-based models are compared, and as a result, a CNN model is selected due to its remarkable accuracy of 99.44%. In Reference [27], an LTSM-based SMS spam model is proposed with 6,000 features and 200 LSTM nodes, achieving an accuracy of 99.01% with three different word embedding techniques: Word2Vec, WordNet, and ConcepNet [6]. LSTM is one of the best-known models for content-based spam detection [11]. In the real world, many emails contain not only text but also image. In Reference [38] the authors propose CNN for image extraction and LSTM for text extraction solutions and provide proof showing that it is a feasible solution.

2.5 Real-time Spam Filter

In general, spam detection solutions use predictive analysis modeling. Predictive analysis refers to using historical data, then applying predictive ML algorithms to create a mathematical model. Afterwards, the model is applied to new data to predict what will happen further on.

2.5.1 Concept Drift in Machine Learning. There are three key components in predictive analysis modeling: the data, the algorithm, and the model [40]. Theoretically, predictive modeling can be presented as $y = f(x)$, in which the relationships between input and output data do not change. However, in the real world, the relationships between the input and output data can change over time, which causes the predictions made based on models that were trained on older historical data to become inaccurate. The theory of concept drift in ML and data mining refers to this type of change [40].

2.5.2 Changing Data Forms. The change to the data is categorized into three main forms: a gradual change over time, a cyclical change, and a sudden/abrupt change. Because both gradual and cyclical changes are patterned changes, such as time series forecasting and predictions on streaming data, the problem of concept drift can be explicitly tested and addressed. However, since cybersecurity detection including spam detection is a sudden unpredicted form of data change, more research is needed to deal with the concept drift issue in this context [26].

2.5.3 Concept Drift Detection. Different concept drift detection solutions may be applied depending on the form of data change. First, a manual intervention is involved when concept drift is suspected. The second solution introduces a periodical re-fit on the static model, which is being updated monthly or yearly based on the periodically collected data. It may also involve a back-testing of the model to select a suitable amount of historical data to determine the period of re-fitting. Being more efficient, the third solution uses the existing model as the starting point for updating the model to fit the most recent data rather than discarding the static model completely. This

approach is suitable for most ML algorithms, especially for neural networks, since it performs a periodical re-training process and updates the model by adding more source data. As for the fourth solution, for certain algorithms, the importance of the input data is weighed and adjustments can be made based on the change of the importance of the input data. The fifth solution is to constantly switch between models, especially when a model cannot be easily updated by adjusting weights and parameters. Last, learning the changes by adding a boost-layer into the model based on newly received data, especially for the first two forms of data changes, the new model allows anticipation of changes and reduces the effect of changes of input data according to the original model [26, 40].

2.5.4 Spam Filter Concept Drift. Although various ML/DL algorithms for spam classification and clustering have been studied, most of them do not provide a dynamic update solution to deal with non-stationary data. Since spam data is non-stationary and most existing spam classification solutions process spam data as presumed stationary data, we focus the remainder of our literature review on real-time solutions for general non-stationary data.

Two main approaches are reviewed in Reference [2] on non-stationary data. One is to apply an incremental method using ML/DL algorithms. The model is updated every time it proceeds with a new training instance. Incremental versions of SVM and neural networks are incrementally learning methods. Another approach is to use periodic re-training with a batch algorithm by simply buffering the relevant data and retraining the model periodically. To achieve a high-performance result, parallelizing the ML algorithms or evaluating new ones by using parallelization methods is proposed on a conceptual level in this article. The programs can be divided into multiple independent tasks with the scenario that when one thread is blocked another thread can run [2]. The author in Reference [1] examines and studies new approaches for classification of the non-stationary dataset German Credit Data using a one-dimensional CNN+LSTM algorithm for real-time data classification. The performance of both algorithms is superior to those of other similar classifiers. LSTM also outperforms CNN on classifying streaming data [1]. A real-time intelligent system is reviewed in Reference [15] to allow adaptive reaction to real-time and unexpected events. A layered architecture integrating a real-time scheduler with **Artificial Intelligence (AI)** has been designed in a simulated information access system to broadcast information to mobile users. The results show that the application of artificial intelligence to the real-time scheduler improves the adaptiveness, responsiveness, and performance of the system.

3 PROBLEM STATEMENT

According to various statistics, existing spam filters help to reduce spam at a certain level. However, problems such as message delivery delay, complex administrator configuration interfaces, and misclassified spam and legitimate email remain unsolved [29]. Recently, ML applied to textual and non-textual attributes has been used to differentiate spam from legitimate content [22]. Text mining has become an important component of the anti-spam solution. Many different classification algorithms have been used and evaluated, including NB, Bayesian Net, SVM, Decision Trees, Random Forests, and so on, with very high accuracy results, underscoring that ML-based methods are successful in spam classification [9, 16]. However, these experiments are based on specific or proprietary datasets [12, 16]. Many ML-based spam filtering approaches [13, 31] suffer from the limitation of manual feature engineering, which does not scale up. Identifying prospective features for accurate classification requires prior knowledge and domain expertise. Furthermore, the selection of the features needs to be reassessed based on criteria such as information gain and feature importance. The choice of features greatly impacts classification accuracy. The iterative trial-and-error manual process is expectedly time-consuming. Many research results reveal that ML spam filter algorithms take a long time to complete the detection task [13, 31].

DL algorithms use multiple layers to transform raw input data into an abstract form. Researchers see a need to apply DL to spam filtering specifically to utilize its numerous processing layers and levels of abstraction to learn the representations of data. When making a semantic analysis where relative syntactic structures are involved, research results demonstrate that the DL produces better results than traditional ML when classifying millions of e-mails [19, 21, 24]. However, as spam becomes more sophisticated, there is always a need for more sophisticated solutions on spam detection automation. In this article, we first compare several ML algorithms on email datasets of various sizes and report on the performance of different data representation methods. We then propose a better content-based spam filter algorithm that combines CNN and LSTM models in Keras.

Most of the current spam filtering products analyze the content of emails and label spam only as junk mail and expect the recipients to decide on the authenticity of the source [5, 33, 39]. Three problems have appeared related to this approach. First, the fact that all suspected spam has already proceeded through an email server causes an unnecessary waste of system and network resources. Moreover, it adds extra load to content-based spam classifiers and decreases accuracy and precision and is more likely to increase the FPR. Last, it is very difficult for non-technical users to verify the authenticity of the source of an email. Moreover, most of the spammers obfuscate their email content to circumvent the spam filters. This problem with content-based spam filter is further exacerbated when filtering spam such as emails with few or no words, and emails with one or more images. Sometimes the content of email could be just the URL of a phishing website [39]. Therefore, we require a solution for preventing unsolicited emails using other aspects of the email that spammers cannot manipulate. Currently, there are no solutions that are able to lead to the complete eradication of spam. Considering the issues discussed so far, we propose a novel framework that includes both a data-driven context-based hybrid DL classifier and rule-based knowledge engineering solutions. Our classifier considers the path traversed by an email and the reputation of the relays that spammers have no control of. Though spammers can spoof the IP addresses of the relays present at the beginning of the path traversed by an email [27, 33], IP addresses recorded by the recipient domain's inbound mail servers can be trusted. Moreover, IP addresses recorded by any domain that has ability to identify their outbound mail servers can also be trusted. There are many rule-based knowledge engineering techniques that are included in the rule-based components, such as malware signature, DNS blacklist, blacklist, white-list, grey list, and so on [27, 39]. A block diagram of the proposed novel framework is presented in the next section. One more problem we found from the literature review is the inability of the current spam filtering techniques to effectively deal with the concept drift phenomenon considering the ever-increasing email traffic and data. [2] The majority of the existing spam filters are unable to incrementally add or delete features without re-creating the model to keep track of current trends in spam filtering. As a result, these filters do not possess the capacity to incrementally learn in real-time. DL methods work on feature representations and constantly learn and fine-tune the features [2]. In Reference [15], it addresses the incremental versions of SVM and neural networks. Parallelization methods are proposed on a conceptual level [1]. These methods overcome the weakness of conventional classification techniques, such as a lack of experience in dealing with the real-time environment, data streams, and concept drift, through a parallelization method and a combination of CNN and LSTM DL models.

In summary, six problems from the existing research works are shown as follows:

(1) The current spam classification accuracy is insufficient to deal with the massive non-stop flow of inbound emails.
(2) Manual feature engineering relies heavily on domain knowledge and has a slow reaction that does not meet the requirements of "Zero-day" spam detection.

(3) The majority of the existing spam filters are unable to incrementally add or delete features without re-creating the model to avoid the concept drift phenomenon and keep track of current trends in spam filtering.
(4) Spammers can spoof the IP addresses, which leads to big challenges in spam detection.
(5) There is no all-rounded solution that prevents spam in real-time.
(6) All suspected spam are still delivered to the user's mailbox junk mail folder awaiting manual inspection, which leads to low productivity.

4 PROPOSED SOLUTION

Based on the deficiencies discussed in the previous section, a novel framework is proposed to address each of the six identified problems.

(1) A daily updated hybrid DL/ML intelligent content-based spam engine is employed as the top layer to address the first three problems.
(2) Spam and malware signatures are updated on a daily basis to address the fourth problem.
(3) A rule-based non-content-based filter component as the bottom layer works in combination with the top layer to address the last two problems in an efficient and dynamic fashion.

The proposed framework aims to detect spam in real-time with high accuracy before the emails are delivered to the users' inboxes. Existing related works either focus on rule-based non-AI solutions or on content-based data-driven AI algorithms to classify spam. No solution analyzes all the parts of email gathering information to meet the goal of low FPR and FNR spam detection. Moreover, most solutions are offline, lacking the functionality to analyze online email data to incrementally update the model, which is required in the real world. To achieve this goal, we propose a layered online real-time spam filtering solution to offer the best of both worlds. It starts at the layer 1 filter, a rule-driven filter for the non-content-based unit, which detects and filters "sure spam" and "sure ham." Therefore, it minimizes the number of emails to the data-driven layer 2 filter on the content-based filter stage. With daily updated spam and malware signatures from the global supports to the layer 1 rule-driven filter, it provides the functionality of a more dynamic and flexible filter for non-stationary spam attacks. Based on this rationale, the accuracy, and efficiency of real-time spam classification will be improved. Besides, based on the literature review, a DL neural network is an incremental learning method that can overcome the weakness of conventional spam classification techniques and is viable to cope in a real-time environment. The layer 2 filter is a data-driven content-based filter that combines CNN and LSTM DL neural networks. It is designed to handle evolving data streams and concept drift while producing in real-time high accuracy results. Three major contributions stem from our research: First, we propose a new hybrid layered rule-based non-AI solution and an email content-based AI solution; second, for the content-based AI solution, we propose a stacked DL-based solution to overcome the weakness of the traditional ML solutions; last, our framework provides online spam detection that can handle incoming emails in real-time and produce accurate results. A high-level conceptual block diagram is illustrated in Figure 1. We provide in Sections 4.1 and 4.2 a detailed description on the two layers, which consist of two major units of our solution: a spam filter learning unit and spam filter execution unit combined through a parallelization method.

4.1 Online Spam Filter Learning Unit

The learning unit trains and updates the most suitable classification model on the input batches of a continuously updated historical email corpus. The source of data is the collection of the full email message packets including the whole email message header and the contents of the email body.

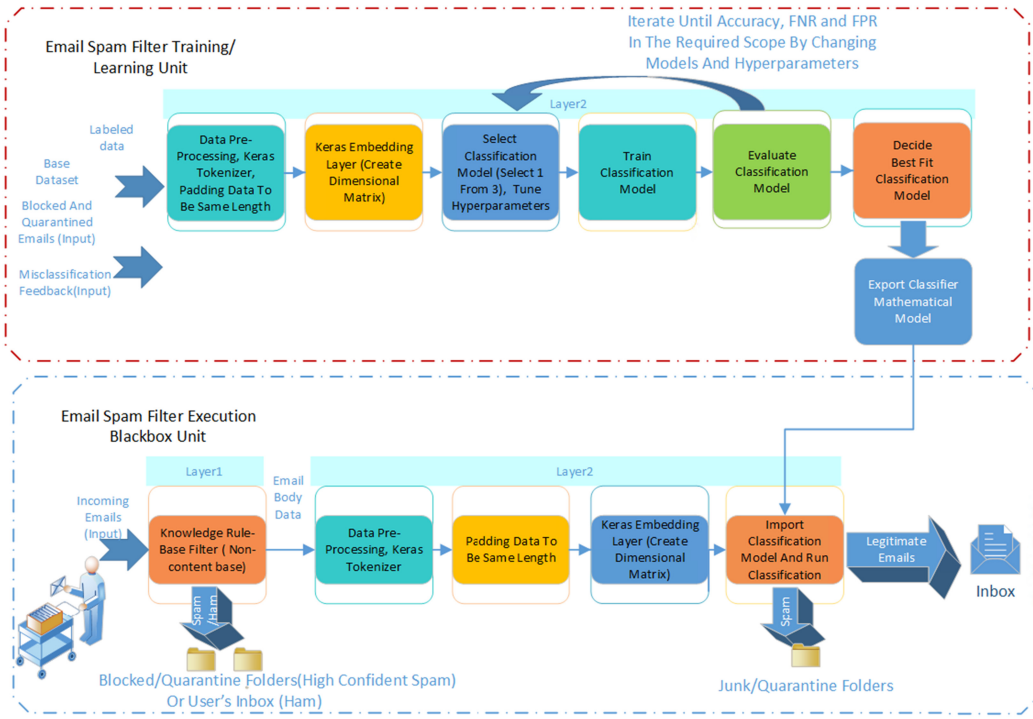


Fig. 1. Overall structure of the proposed Real-time Spam Filter Framework.

Although CNNs are usually used in modeling problems related to spatial inputs like images, CNNs achieve very good results on problems such as document classification. LSTMs are used in modeling tasks related to sequences predictions. They are widely used in **Natural Language Processing (NLP)**-related tasks, such as machine translation and sentence classification and generation. According to existing literature works, solely using LSTM does not produce an optimal result. Hence, we introduce a more sophisticated model by combining CNN with LSTM. The CNN+LSTM architecture involves using CNN layers for feature extraction on input data combined with LSTMs to support sequence prediction.

Hybrid classifiers are those that classify over multiple iterations, rather than classifying by comparing the output of different processes. Our goal is to leverage hybrid classification to maximize performance.

Through a process of data pre-processing, feature extraction, modeling, and model exporting, the modeling function block provides several hybrid DL options for the content-based classifier. The best classifier model is selected as the output of the learning unit and is exported to the execution unit. Algorithm 1 demonstrates a detailed procedure of the above-mentioned learning unit. Before the continuous training of incoming labeled datasets, $D_{labeladd}$, an initial default model, $Model_{init}$, is trained on a given base dataset, D_{base}. We set the initial default model to be CNN+LSTM. During the model selection phase, three choices of model are provided: (1) CNN with one layer of LSTM (CNN+LSTM), (2) two layers of LSTM (2LSTM), and (3) CNN with two layers of LSTM (CNN+2LSTM). A model is selected when (1) the classification accuracy of the trained $Model_{init}$ reaches 98.5%, (2) FPR is less than or equal to 0.6%, and (3) FNR is less than or equal to 0.6%. Otherwise, we keep tuning the hyper-parameters until the model reaches these thresholds. In the worst-case scenario, where all three models fail to reach this threshold, we set

ALGORITHM 1: Learning Unit

```
Input: D_base, D_labeladd
Output: CurrentModel
Initialize Model_list, M_i, i ∈ {1, 2, 3};
Train Model_init = CNN+LSTM on D_base;
for each incoming D_labeladd do
    Data pre-processing, tokenization, and padding the data to the same length;
    Split D_labeladd into training, validation, and testing set;
    Train each M_i (keep tuning hyperparameters to obtain the best result);
    CurrentModel = The best M_i with accuracy >= 98.5%, FPR <= 0.6% and FNR <= 0.6%;
    if none satisfies condition then
        CurrentModel = Model_init;
    end
end
Update CurrentModel;
Export CurrentModel;
```

the current best model back to the initial default model. Hence, a chosen model would be exported from the learning unit.

4.1.1 Set and Check Variables and Training Set. A text variable is used for email content and a binary variable is used to label spam (1) or non-spam (0). In Algorithm 1, every incoming $D_{labeladd}$ is split into a training, validation, and testing set before training each M_i and consequently produces an accuracy result based on the testing set. The optimal split ratio is further examined in Section 5.3.2.

4.1.2 Data Pre-processing. The data pre-processing phase aims to prepare a suitable dataset for classification modeling. In this phase, various activities including stop word removal, stemming, lemmatization, and normalization with noise removal are performed for the subject and body of the emails.

4.1.3 Keras Word Embedding Layers for DL Model. Following the data pre-processing phase is a feature extraction phase. This phase classifies words based on their term frequency using the Term Frequency Matrix technique [28]. The text is tokenized and an embedding matrix is created. This is done to provide an unbiased context for the model, and thus, to be able to classify more difficult sentences as well. The procedure details are as follows:

(1) Tokenization
The email body contents are tokenized into individual tokens by transforming text to a sequence of integers. The purpose of this action is to quantify text input to a convenient setup.
(2) Text padding
After tokenization, all documents' contents are converted to sequences of same lengths (i.e., same number of words). A fixed document length is specified. Documents with length lower than the specified length are padded with zeros. For documents with length greater than the specified length, tokens are deleted from the beginning of the sequence until the necessary length is reached.
(3) Modeling embedding layer
The pre-processed tokens are sent to the embedding layer to generate an embedding matrix, which stores the features of each token and serves as an input to the trainable deep classifiers.

We choose a Keras embedding layer with 2,000 vocabularies and a vector space of 128 dimensions, in which words are embedded. Word embeddings use an efficient and dense representation, in which similar words have a similar encoding. An embedding is a dense vector of floating-point values.

4.2 Online Spam Filter Execution Unit

From the high-level conceptual block diagram shown in Figure 1, we can see in the execution unit that the first few function blocks are the same as the learning unit. They are rule-based non-content-based filtering, data pre-processing, and feature extraction. Then, the model exported from the learning unit is implemented as a spam classifier. The execution unit detects spam in real-time before the emails are delivered to the users' inboxes. The input of this unit consists of all inbound emails. All data flows through a rule-driven function block in the non-content-based filter. The source of data is the collection of the full email message packets, including the whole email message header and the contents of the email body.

ALGORITHM 2: Execution Unit

```
Input: incoming email samples, CurrentModel
Output: Spam/Ham, D_labeladd
Initialize the knowledge-based Rule_set, R = r_1, ..., r_n;
for each incoming sample do
    if Sample meets requirements in R then
        Return all sure spam and sure ham;
    end
    else
        Run CurrentModel;
    end
    Return Spam/Ham;
end
Append all samples to D_labeladd according to the feedback from the email server;
```

Algorithm 2 specifies the procedure used by the execution unit. For every incoming email, the layer 1 filter is first used by iterating through the existing rule-based classifiers, R. For each spam detected, the layer 1 filter executes all the knowledge engineering rules to decide whether to block or quarantine it. For each ham detected, both layer 1 and layer 2 filters are skipped and **Mail Transfer Agent (MTA)** transfers this ham to the user's inbox directly. When a sample does not get filtered out by layer 1, it is sent to layer 2, the content-based filter, and uses the exported model from the learning unit to perform classification. The output of layer 2 is a classification result of spam or non-spam for each email. It then passes the result along, allowing the MTA to decide whether to move the email to the users' inbox, junk mail folder, or quarantine folder. Finally, all samples from user feedback (e.g., confirmation of spam) are used to produce or update the labelled dataset, $D_{labeladd}$, which is one of the inputs of the learning unit.

5 PERFORMANCE EVALUATION

We have described each function block of our proposed framework in detail in Section 4. The spam learning unit offers functionality to allow for easy selection and swapping of the classifier algorithms. In this section, we focus on the analysis and evaluation of experimental results of the text binary classifiers in the content-based spam filter unit. Our experiments show that the CNN+LSTM and CNN models are likely to deliver the best results. We limit our implementation of

Table 1. Efficiency Comparison on Execution Time and Speed

Models	Execution Time (sec)	Speed (msec/step)
CNN+2LSTM	17	10
CNN+2LSTM 2Dense	19	11
CNN+LSTM	3	2
CNN+LSTM 2Dense	10	6
2LSTM	14	8
2LSTM 2Dense	11	6
LSTM	18	10
LSTM 2Dense	17	10
CNN	3	2

functional testing to eight performance metrics, and we perform efficiency analysis mainly on the processing time for these DL classifiers and common ML classifiers. We compare the selected DL classifiers against existing solutions. Furthermore, we also conduct experiments on the selected DL classifiers with different hyper-parameters to show their effects on the performance of the classifiers. As an overall quality measurement, we have measured all eight common performance metrics for the results of the average of three runs for each experiment set.

5.1 Performance Measurement on Spam Detection

The common performance evaluation measures of spam detection are accuracy, precision, recall, F-Measure, specificity, classification error, FPR, and FNR [12, 16]. We use a confusion matrix to visualize the performance of the algorithms. The confusion matrix uses two columns to represent the predicted classified items and two rows to represent the actual classified items.

5.2 Comparative Evaluation

5.2.1 Dataset. The dataset we used in the experiment is a publicly available dataset first described in the year 2006 [17]. The spam messages in this dataset are the SpamAssassin corpus, which contains hand-labeled spam messages contributed by Internet users. There are 5,728 emails in total, which includes 1,719 spam and 4,009 non-spam (hams). The dataset contains two fields, which consist of the textual email content and a binary variable with 1 for spam and 0 for non-spam, respectively. We have selected this dataset, as it is currently the most realistic publicly available data set for testing spam detection schemes.

5.2.2 Experimental Setup. We perform extensive experiments on the layered CNN+LSTM model and evaluate the performance using Keras 2.24 API using a TensorFlow 1.13.1 back-end in Python 3.7 (64 bit). This is implemented on a Windows 10 computer with a 5th generation Intel CORE i5 processor.

5.2.3 Experiments on Evaluating Efficiency: Execution Time, Resources, and Computing Cycles. We have run an experiment to evaluate the efficiency of nine DL models. The results for the processing time of the nine classification algorithms are given in Table 1 and Figure 2.

Table 1 shows the average processing time of each model. It demonstrates that the processing time of the CNN+LSTM and CNN classifiers are the shortest, and the processing time of the CNN+2LSTM classifier using two dense layers is the longest. Since a fair amount of the emails are filtered out by the rule-based layer, the average processing time for the classification can be greatly reduced. From these results, we see that both CNN+LSTM and CNN are the most time-efficient classifiers.

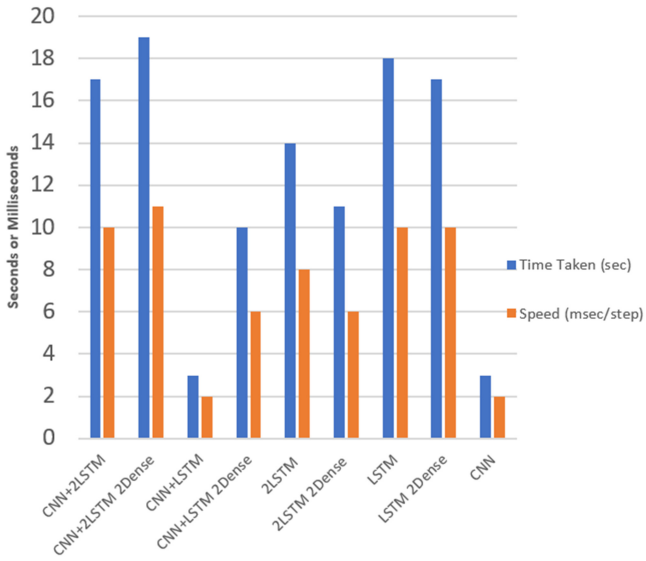


Fig. 2. Efficiency comparison of nine DL models on execution time and speed.

Table 2. Performance Comparison between Proposed Hybrid Models and Other DL and ML Methods

ML/DL Models	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
LSTM	97.61%	96.02%	93.92%	94.96%	98.78%	2.39%	6.08%	1.22%
CNN	99.19%	98.3%	98.3%	98.3%	99.64%	0.81%	1.7%	0.54%
CNN+LSTM	99.36%	98.54%	98.78%	98.66%	99.54%	0.64%	1.22%	0.46%
CNN+2LSTM	98.78%	98.03%	96.84%	97.43%	99.39%	1.22%	3.16%	0.61%
RF	96.97%	99.73%	87.8%	93.38%	99.92%	3.03%	12.2%	0.08%
KNN	91.91%	93.46%	71.77%	81.19%	98.39%	8.09%	28.23%	1.61%
LR	98.72%	98.53%	96.17%	97.34%	99.54%	1.28%	3.83%	0.46%
DT	84.93%	61.74%	100%	76.35%	80.09%	15.07%	0%	19.91%
SVM	95.06%	98.26%	81.1%	88.86%	99.54%	4.94%	18.9%	0.46%
GNB	95.64%	97.25%	84.45%	90.4%	99.23%	4.36%	15.55%	0.77%

5.2.4 Performance Comparison on Proposed Algorithms versus Existing DL and ML Classification Algorithms. We use the same performance metrics to compare all proposed algorithms in this article against existing ML algorithms. We select LSTM, CNN, CNN+LSTM, CNN+2LSTM, RF, KNN, LR, DT, SVM, and NB as the algorithms for comparison. All algorithms are implemented in Python 3.7. The results are given in Table 2 and Figure 3.

Table 2 and Figure 3 describe all the evaluation results and performance measurements of the traditional ML models and DL models along with the proposed approaches on the same spam dataset. The CNN+LSTM model outperforms the DL models and other ML models with an accuracy of 99.36%, while CNN and CNN+2LSTM reach second and third places, respectively. In general, the choice of metric depends on the business objective. In the context of spam classifier, FPR is usually more important compared to FNR in business scenarios, since spam going to the inbox (FNR) is more acceptable than non-spam getting caught by the spam filter (FPR). As a result, relative metrics such as precision and specificity are also important for optimization. Table 2 shows the score

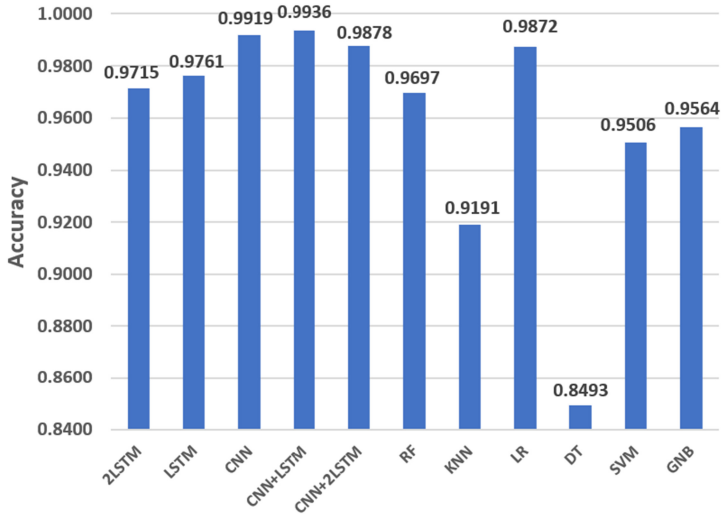


Fig. 3. Performance comparison between the proposed hybrid models and other ML methods.

of precision, specificity, and FPR, in which the **Random Forest (RF)** algorithm has the best performance and CNN+LSTM has the second-best performance. However, RF's recall and FNR shows poor performance compared to all the tested DL models. Hence, methods like CNN+LSTM with 1 dense layer and CNN achieve good accuracy simply and effectively. Although some ML algorithms, like RF, provide an acceptable level of accuracy and slightly higher precision, specificity, and FPR, they are not able to "self learn," which is considered as an important feature of the DL methods. Our experimental results indicate that the performance of CNN+LSTM and CNN+2LSTM is better than that of other algorithms except for RF and CNN, which are very close in performance. If we have a higher demand for accuracy and time overhead, then CNN+LSTM can provide a balance between all the performance metrics and processing time, hence CNN+LSTM is better suited for data stream classification tasks.

5.2.5 Receiver Operating Characteristics (ROC) and Area Under the Curve (AUC). In addition to accuracy, FNR and FPR also play significant roles in evaluation. ROC and AUC are suitable for analyzing the performance of classification algorithms with binary outcomes. An ROC chart can be considered as the average value of the sensitivity for a test over all possible values of specificity or vice versa. Additionally, ROC curves assess predictive behavior independent of error costs or class distributions. Since the ROC curve can provide a better look at where different classifiers minimize the cost, we set up ROC analysis to compare the performance of the algorithms. Any sort of data that can be fed into appropriate classifiers can be subjected to ROC curve analysis. We first experiment with ROC for all selected ML models. Figure 4 shows the ROC curves for the DL and ML algorithms.

We group variants of multiple ROC curves on the same plot. A better classifier has an ROC curve that rises steeply to the top-left corner, that is, it will correctly identify many positive instances without misclassifying negative instances. Figure 4 demonstrates that both CNN+LSTM and CNN+2LSTM stand out in the top-left corner. AUC is useful as a single number summary of classifier performance. According to Figure 5, both CNN+LSTM and CNN achieve a score of 0.992, which is considered in the scope of excellent algorithms and further verifies their feasibilities from the ROC curve performance.

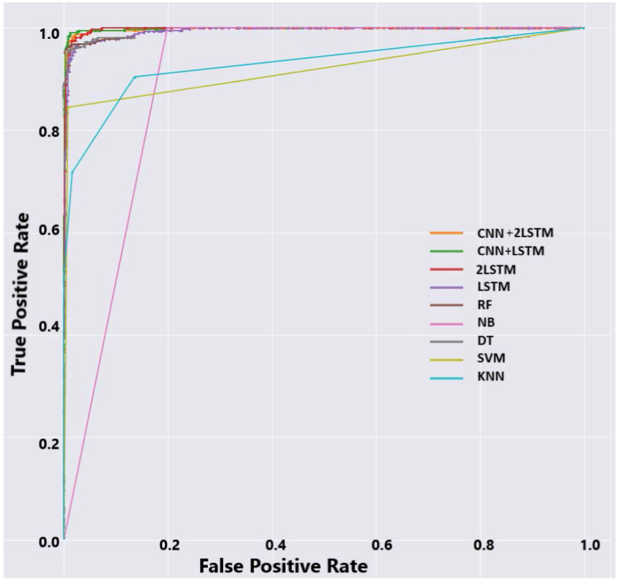


Fig. 4. ROC for our proposed models and all other ML models.

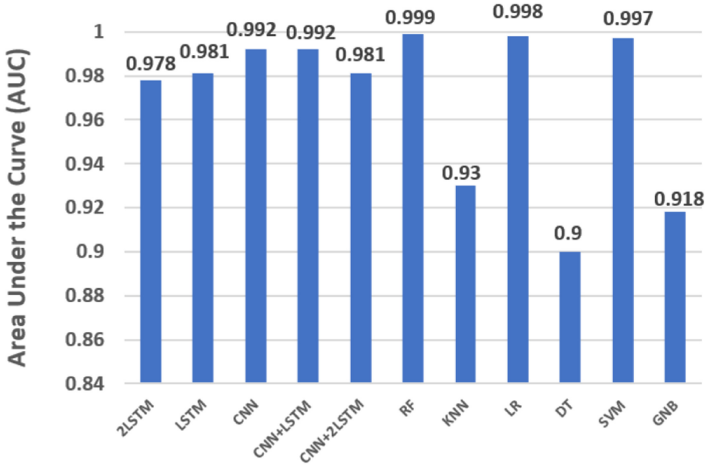


Fig. 5. AUC for our proposed models and all other ML models.

Binary classifiers are mostly evaluated using sensitivity and specificity, and performance is illustrated using ROC plots. However, ROC plots can be misleading when applied in imbalanced classification scenarios. Since our dataset is balanced, we use ROC for analysis. Precision-recall plots, shown in Figure 6, provide an accurate prediction of classification performance, because they evaluate the fraction of true positives among positive predictions. With a balanced dataset, the areas under both the ROC curve and the Precision-Recall curve are the same. Hence, we create both curves to illustrate our previous conclusion, namely, that our dataset is balanced. Since our proposed framework will be used in production for real-world online data, which may be unbalanced, so we use both curves to evaluate models to decide which should be engaged in the

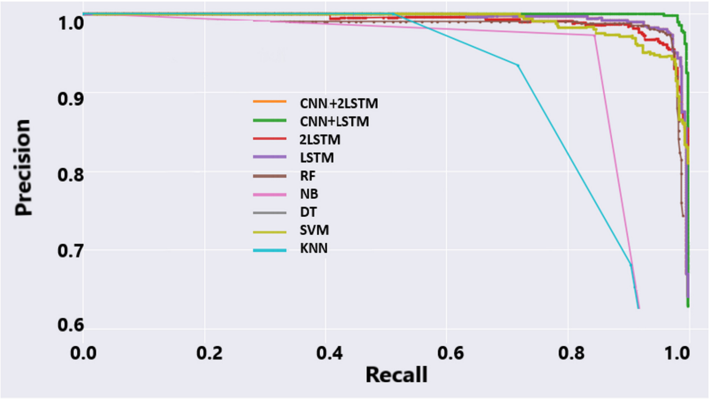


Fig. 6. Precision-Recall Curve for all models.

framework. Overall, the experimental results of ROC and AUC demonstrate that the proposed hybrid model, CNN+LSTM, shows an enhanced classification accuracy compared to base classifiers and ensemble classifiers. It yields the highest accuracy of 99.36%. Neural networks and many statistical algorithms are examples of appropriate classifiers according to the literature review, while approaches such as decision trees, KNN, and NB are less suited. Thus, we use an ROC chart to further illustrate the performance of the CNN+LSTM model. In conclusion, the performance of the CNN+LSTM model is indeed better than other models that we have studied.

5.2.6 Performance Evaluation on Eight Different DL Classification Algorithms. To demonstrate that the proposed model is the right choice, we run an experiment on a different combination of algorithms. We explore additional hierarchical learning capacity by adding more LSTM and dense layers. The accuracy, precision, sensitivity, specificity, F1-score, FNR, FPR, and classification error for each of the classification algorithms are given in Figures 7(a) and 7(b).

The overall classification accuracy rate corresponds to the proportion of observations that are correctly classified. Therefore, determining the classification accuracy is the first step in assessing the performance of the models. In contrast, the classification error rate is defined as the proportion of observations that are misclassified. Error rate = 1− accuracy. The raw classification accuracy and error can be easily computed by comparing the observed classes in the test data against the classes predicted by the model. Precision denotes the fraction of email correctly classified as spam out of all email classified as spam while recall denotes the fraction of email correctly classified as spam out of all email that are indeed spam. The F1 score function is denoted as a weighted average of both precision and recall. Figures 7(a) and 7(b) illustrate these values for all DL classifiers, out of which CNN+LSTM has the highest value on all eight performance metrics. Furthermore, Figure 7 shows that most of the algorithms' performance metrics are improved by adding another dense layer except for 2LSTM. The use of two dense layers is slighter better than 2LSTM with one dense layer. Our experimental results confirm that the proposed hybrid model, CNN+LSTM, provides enhanced classification accuracy compared to base classifiers and ensemble classifiers. It yields the highest accuracy (99.36%) as well as the best performance on all other measures.

5.3 Evaluation of Hyper-parameters Optimization for Deep Learning Models

In this section, we provide the results of an evaluation on CNN+LSTM and CNN+2LSTM comparing different hyper-parameter configurations. The hyper-parameters we have investigated are

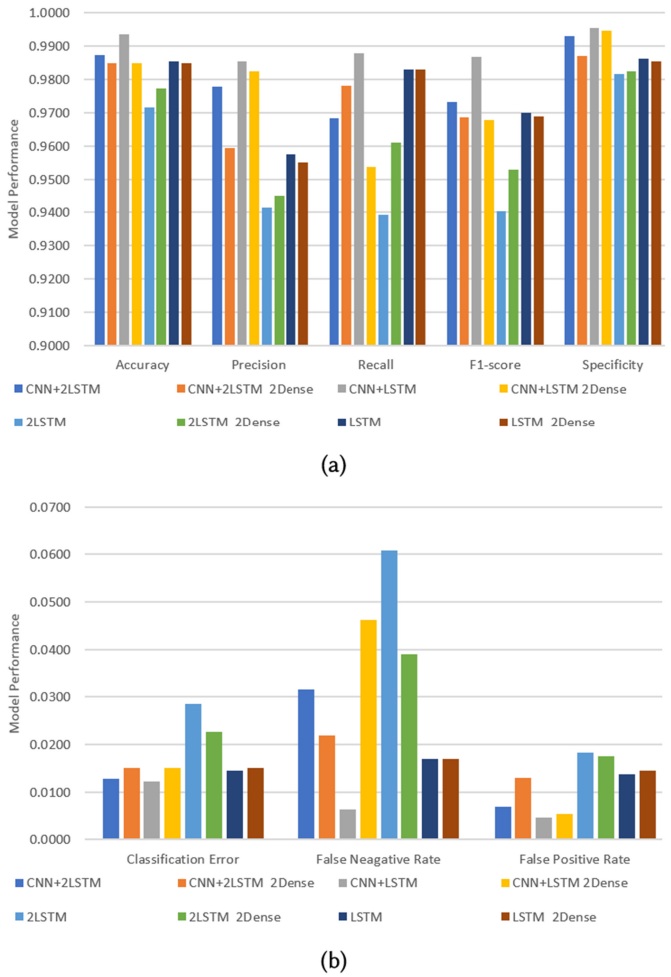


Fig. 7. Performance comparison of results on different DL classification algorithms.

number of dimensions, train/test split ratio, number of epochs, batch size, dropout rate, and activation function.

5.3.1 Word Embedding. It is common to see word embeddings ranging from eight dimensions for small datasets and up to 1,024 dimensions for large datasets. A higher dimensional embedding can capture fine-grained relationships between words but takes more data to learn. To evaluate which dimension setting is optimal, we set the experiment to use the words that are converted into word vectors by applying word embedding layers for CNN plus two layers of LSTM and CNN+LSTM in Keras using eight epochs, a batch size of 32, and a dropout rate of 20% on a 70/30 ratio train/test split. Semantic vectors with dimensions of 64, 112, 128, 144, and 192 are compared to select the optimal model. The results are given in Table 3(a), Table 3(b), and Figure 8. Our results show that a dimensionality of 128 is the best setting in terms of performance on all 8 measurements.

5.3.2 Dataset Split Ratio. There are 5,728 emails in total. The full dataset contains roughly a 70/30 ratio for the ham and spam, respectively. The total training set size is 4,296, in which the

Table 3(a). The Effect of Word Embedding Dimensions on CNN+2LSTM

#of Dimensions	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
64	97.85%	96.06%	94.89%	95.47%	2.15%	5.11%	98.78%	1.22%
112	98.6%	96.85%	97.32%	97.09%	1.4%	2.68%	99.01%	0.99%
128	98.72%	97.79%	96.84%	97.31%	1.28%	3.16%	99.31%	0.69%
192	98.43%	97.06%	96.35%	96.7%	1.57%	3.65%	99.08%	0.92%
144	97.85%	96.29%	94.65%	95.46%	2.15%	5.35%	98.85%	1.15%

Table 3(b). The Effect of Word Embedding Dimensions on CNN+LSTM

#of Dimension CNN+LSTM	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
64	97.56%	93.41%	96.59%	94.98%	97.86%	2.44%	3.41%	2.14%
112	98.43%	95.28%	98.3%	96.77%	98.47%	1.57%	1.7%	1.53%
128	98.49%	99.23%	94.4%	96.76%	99.77%	1.51%	5.6%	0.23%
192	98.02%	93.94%	98.05%	95.95%	98.01%	1.98%	1.95%	1.99%
144	97.61%	91.67%	99.03%	95.2%	97.17%	2.39%	0.97%	2.83%

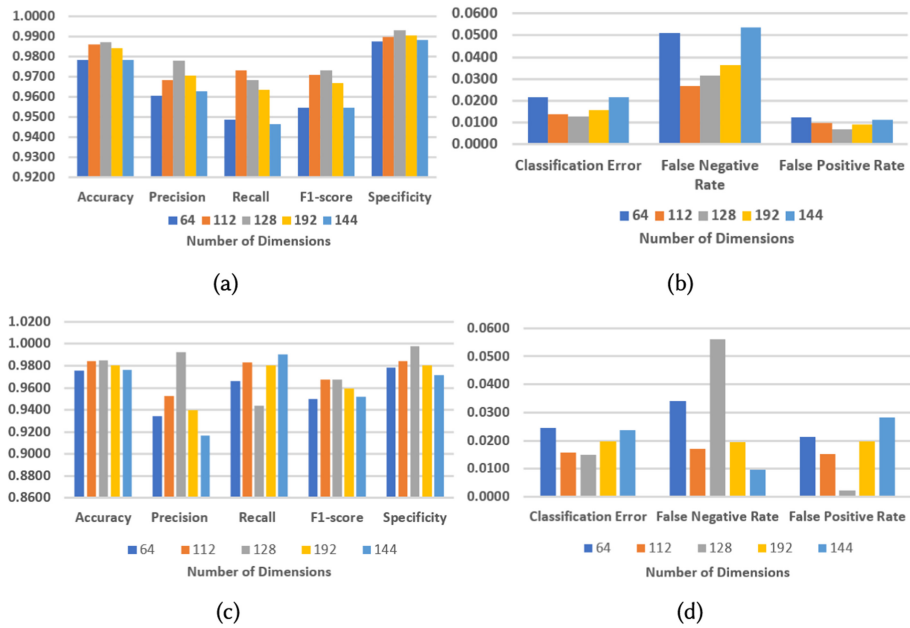


Fig. 8. (a), (b) The effect of word embedding dimensions on CNN+2LSTM. (c), (d) The effect of word embedding dimensions on CNN+LSTM.

total ham size is 3,270 and the total spam size is 1,026. The total size of the test set is 1,432. The percentage of spam in the training set is 23.9%. The results of the classification algorithms using different training and test ratios are given in Tables 4(a) and 4(b), and Figures 9(a), 9(b), 9(c), and 9(d). We have run the experiment on the CNN+LSTM and LSTM models in Keras, using eight epochs

Table 4(a). Performance Comparison on LSTM with Different Train/Test Ratios

Train/Test Ratio	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
80/20	98.08%	95%	95.99%	96.92%	1.92%	4.01%	98.39%	1.61%
75/25	97.63%	91.85%	94.65%	95.81%	2.37%	5.35%	9.73%	2.75%
70/30	98.6%	95.53%	98.78%	97.13%	1.4%	1.22%	98.5%	1.45%
60/40	97.99%	94.65%	95.62%	96.56%	2.01%	4.38%	98.28%	1.72%

Table 4(b). Performance Comparison on CNN+LSTM with Different Train/Test Ratios

Train/Test Ratio	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
80/20	98.25%	97.39%	95.26%	96.31%	99.20%	1.75%	4.74%	0.8%
75/25	98.6%	98.2%	95.91%	97.04%	99.45%	1.4%	4.09%	0.55%
70/30	99.42%	98.31%	99.27%	98.79%	99.46%	0.58%	0.73%	0.54%
60/40	98.21%	94.86%	97.81%	96.31%	98.34%	1.79%	2.19%	1.66%

Table 5(a). Performance Comparison of Different Number of Epochs on CNN+2LSTM

#of Epochs	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
4	98.02%	97.01%	94.65%	95.81%	1.98%	5.35%	99.08%	0.92%
6	97.84%	94.3%	96.78%	95.53%	2.16%	3.22%	98.17%	1.83%
8	98.43%	97.76%	95.62%	96.68%	1.57%	4.38%	99.31%	0.69%
12	98.37%	97.52%	95.62%	96.56%	1.63%	4.38%	99.24%	0.76%

Table 5(b). Performance Comparison of Different Number of Epochs on CNN+LSTM

#of Epochs CNN+LSTM	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
4	98.49%	99.23%	94.4%	96.76%	99.77%	1.51%	5.6%	0.23%
6	99.36%	98.54%	98.78%	98.66%	99.54%	0.64%	1.22%	0.46%
8	99.42%	98.31%	99.27%	98.79%	99.46%	0.58%	0.73%	0.54%
12	97.73%	95.59%	94.89%	95.24%	98.62%	2.27%	5.11%	1.38%

and a dropout rate of 20% on a different training to test ratios: 80/20, 75/25, 70/30 and 60/40. The results are given in Tables 4(a), 4(b), Figures 9(a), 9(b), 9(c), and 9(d). The results show that the 70/30 ratio is the best setting for both CNN+LSTM and LSTM.

5.3.3 Tuning the Number of Epochs. We explore the effect of different numbers of training epochs on performance. The idea is to compare the configurations using summary statistics over several runs and see exactly which of the configurations might perform better on average. The summary of results is listed in Tables 5(a) and 5(b) and Figure 10. We set the experiment to use the words that are converted into word vectors using word embedding layers for both models using a batch size of 32 and a dropout rate of 20% on a 70/30 train/test split with 8, 16, 32, and 42 epochs, respectively. Our results demonstrate that the use of eight epochs is the best setting for both the CNN+LSTM and CNN+2LSTM models.

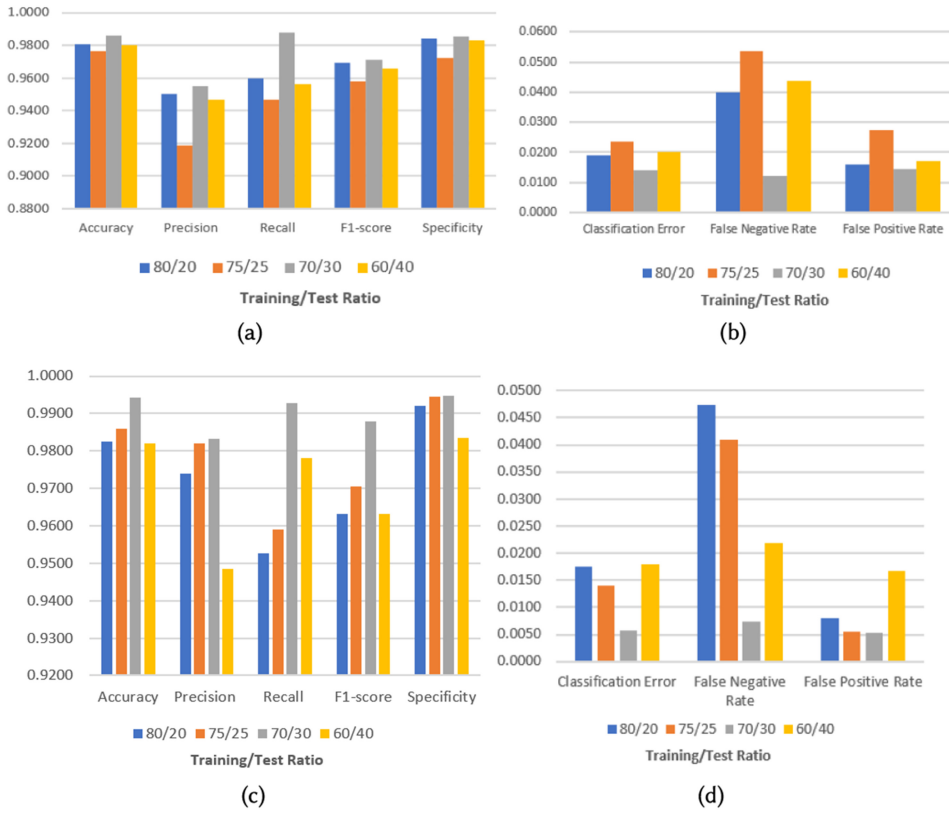


Fig. 9. (a), (b) The effect of different train/test ratios on LSTM. (c), (d) The effect of different train/test ratios on CNN+LSTM.

Table 6(a). Performance Comparison on CNN+2LSTM with Different Batch Size

Batch Size	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
8	97.14%	95.74%	92.11%	93.89%	2.86%	7.89%	98.72%	1.28%
16	98.49%	97.07%	96.59%	96.83%	1.51%	3.41%	99.08%	0.92%
32	98.67%	97.76%	95.62%	96.68%	1.57%	4.38%	99.31%	0.69%
40	98.43%	96.81%	97.66%	97.23%	1.33%	2.34%	98.99%	1.01%

5.3.4 Tuning the Batch Size. Batch size controls how often the weights of the network get updated. In Keras, the batch size must be set as a factor of the test and the training dataset size. In this section, we explore the effect of various batch sizes on the performance. We hold the batch size constant at 8, 16, 32, and 40. Tuning a neural network acts as a means to adjust a tradeoff between average performance and variability of the performance. When a network offers low mean error with low variability, this indicates good performance that is reproducible. The results shown in Tables 6(a) and 6(b) and Figure 11 illustrate that a batch size of 32 provides the best performance for both hybrid models, CNN+2LSTM and CNN+LSTM.

5.3.5 Dropout Rate. The dropout rate is typically set between 20% to 50%. We have run experiments starting with 20%. If the model is over-fitted, then we increase the value. The results

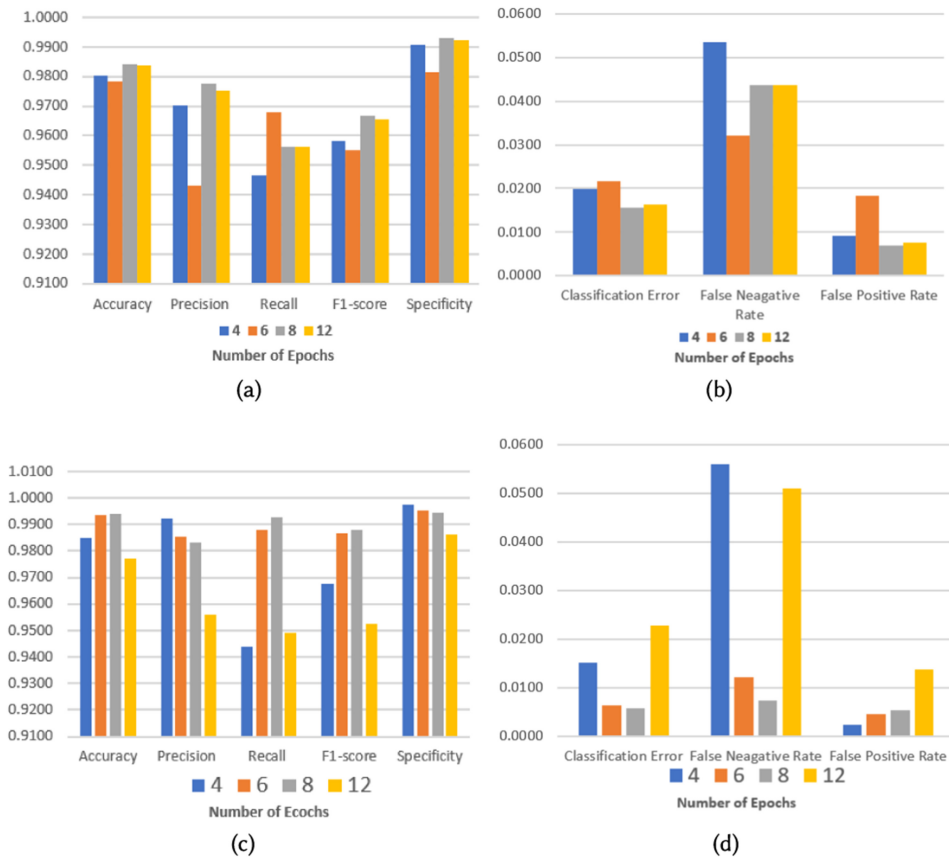


Fig. 10. (a), (b) The effect of the number of epochs on CNN+2LSTM. (c), (d) The effect of the number of epochs on CNN+LSTM.

Table 6(b). Performance Comparison on CNN+LSTM with Different Batch Size

Batch Size (CNN+LSTM)	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
8	98.37%	96.14%	97.08%	96.61%	98.78%	1.63%	2.92%	1.22%
16	98.02%	94.35%	97.57%	95.93%	98.17%	1.98%	2.43%	1.83%
32	99.01%	96.9%	99.03%	97.95%	99.01%	0.99%	0.97%	0.99%
40	96.68%	93.81%	92.21%	93.01%	98.09%	3.32%	7.79%	1.91%

are given in Figures 12(a), 12(b), 12(c), and 12(d), and we conclude that a dropout rate of 20% is the best setting on both 2LSTM and CNN+LSTM.

5.3.6 Choosing the Right Activation Function. To test the influence of activation function on the performance of the algorithm, we select sigmoid and softmax as the activation function options. We test whether the activation function has a notable influence on the performance of all three models, LSTM, CNN+LSTM and CNN+2LSTM. Tables 7(a), 7(b), and 7(c) and Figures 13 and 14 indicate that the activation function has an influence on the performance of the DL models we have compared. Our results show that softmax is better suited to CNN+2LSTM and CNN+LSTM, while sigmoid is better suited to LSTM.

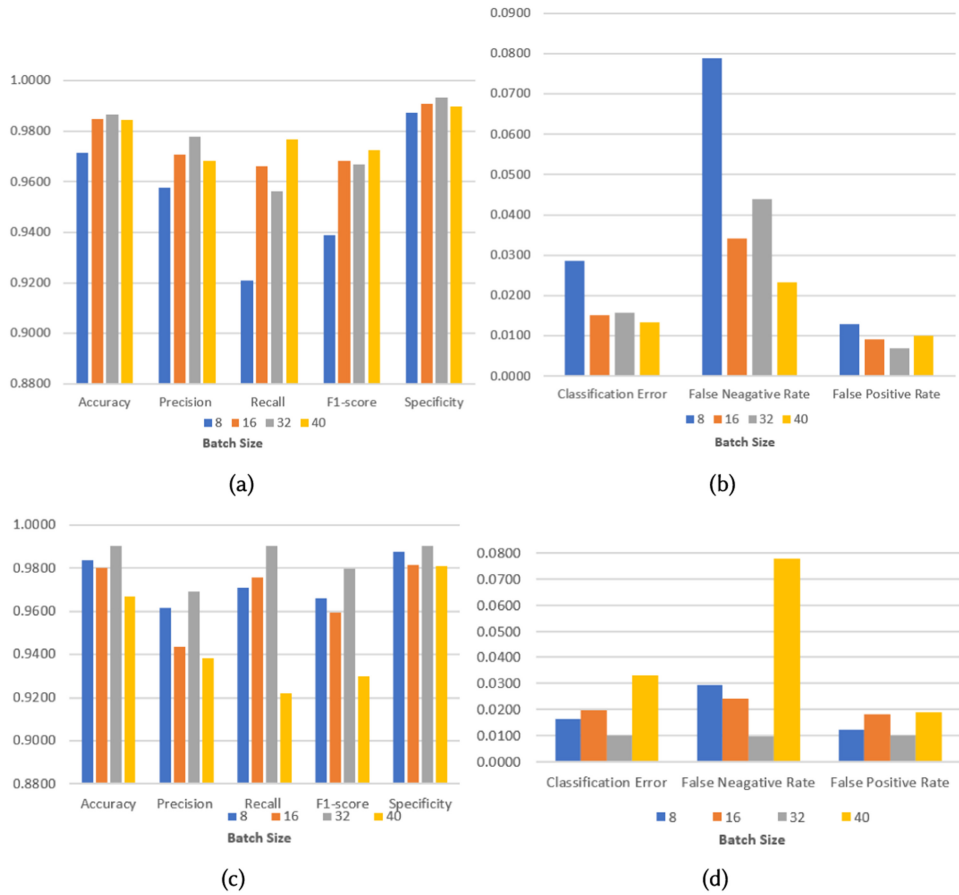


Fig. 11. (a), (b) The effect of different batch size on CNN+2LSTM. (c), (d) The effect of different batch size on CNN+LSTM.

Table 7(a). The Effect of the Activation Function on CNN+2LSTM

Activation Function	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
CNN+ LSTM (Sig.)	98.6%	97.54%	96.59%	97.07%	1.4%	3.41%	99.24%	0.76%
CNN+ LSTM (Soft.)	98.72%	97.79%	96.84%	97.31%	1.28%	3.16%	99.31%	0.69%

Table 7(b). The Effect of the Activation Function on LSTM

Activation Function	Accuracy	Precision	Recall	F1	Cl. Error	FNR	Specificity	FPR
LSTM (Sig.)	97.91%	96.3%	94.89%	95.59%	2.09%	5.11%	98.85%	1.15%
LSTM (Soft.)	97.15%	94.15%	93.92%	94.03%	2.85%	6.08%	98.17%	1.83%

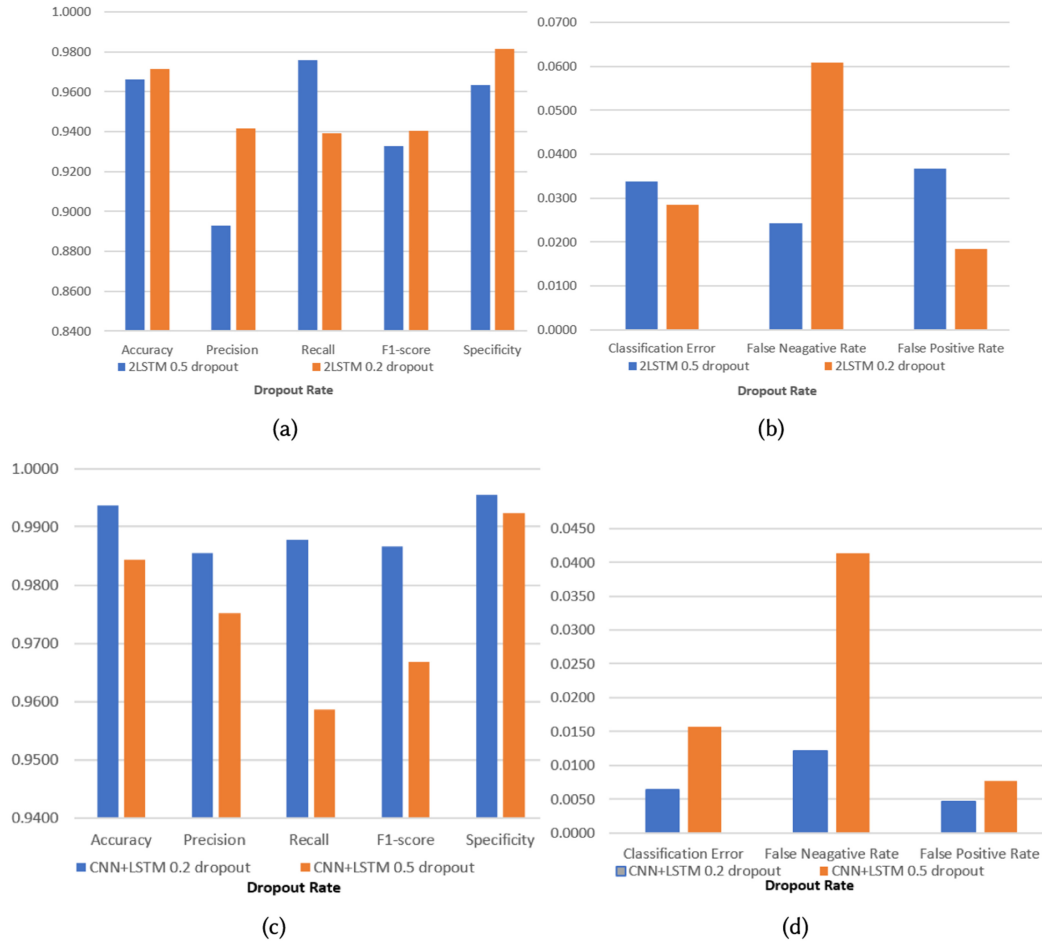


Fig. 12. (a), (b) The effect of different dropout rates on 2LSTM. (c), (d) The effect of different dropout rates on CNN+LSTM.

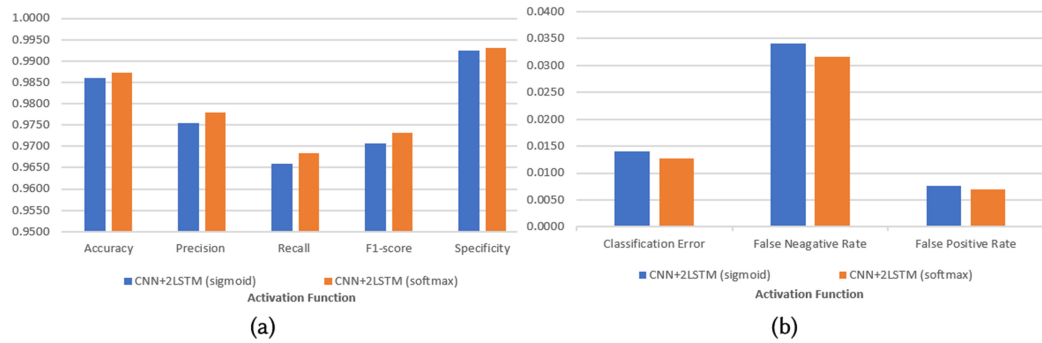


Fig. 13. The effect of the activation function on CNN+2LSTM.

5.3.7 Summary of Evaluation. Overall, our proposed spam filtering approach delivers very reliable results. We have not found any research results on a spam filter that offers a better performance than our approach using the selected dataset. Our proposed hybrid CNN+LSTM

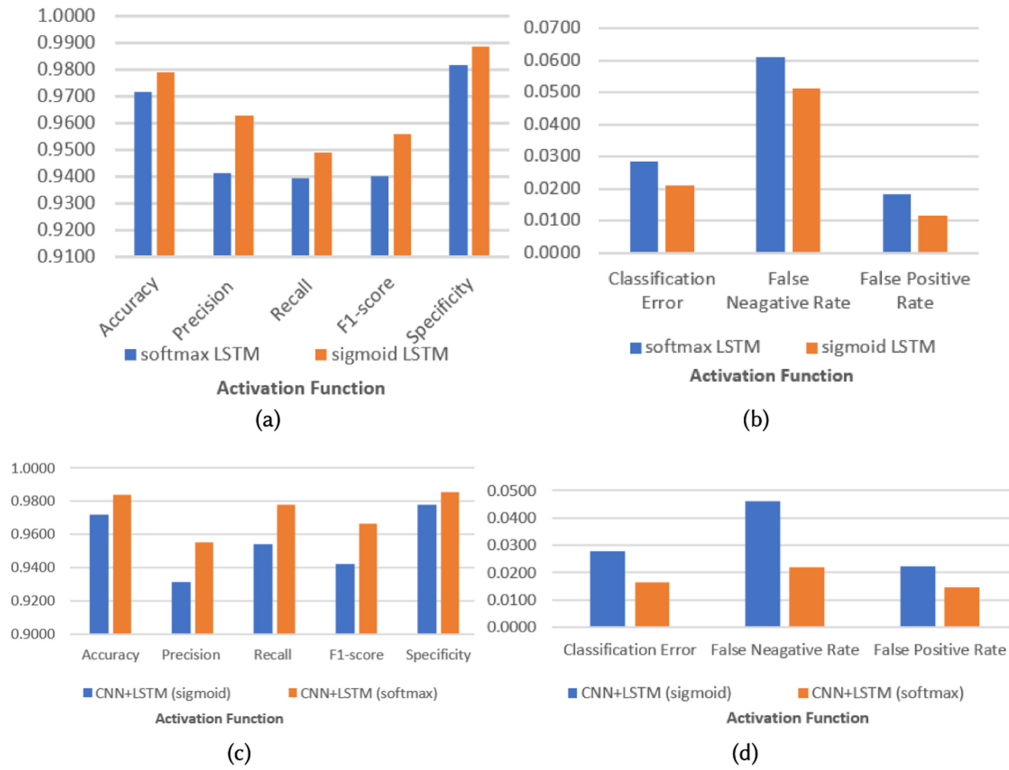


Fig. 14. (a), (b) The effect of the activation function on LSTM. (c), (d) The effect of the activation function on CNN+LSTM.

Table 7(c). The Effect of the Activation Function on CNN+LSTM

Activation Function	Accuracy	Precision	Recall	F1	Specificity	Cl. Error	FNR	FPR
CNN+LSTM (Sig.)	97.21%	93.11%	95.38%	94.23%	97.78%	2.79%	4.62%	2.22%
CNN+LSTM (Soft.)	98.37%	95.49%	97.81%	96.63%	98.55%	1.63%	2.19%	1.45%

approach consistently delivers good performance, namely, above 99% accuracy. If avoidance of false positives is deemed to be more important than accuracy, then CNN+LSTM delivers a slightly smaller FPR while maintaining very high accuracy. Email spam detection is a very challenging problem mainly due to its dynamic changing nature and high accuracy requirements. In this article, we focus on spam detection using a hybrid DL approach. We propose a DL-based approach consisting of CNN and LSTM architectures. Experimental results show that the proposed approach outperforms all other approaches on the selected dataset. We have identified that a dropout rate of 20%, word embedding dimension of 128, and batch size of 32, with a 70/30 train/test ratio and eight epochs with specific activation functions—softmax for CNN+2LSTM and CNN+LSTM, and sigmoid for LSTM—is the best hyper-parameter configuration to achieve high-performance spam detection using our proposed models.

6 CONCLUSION AND FUTURE WORK

In this article, we propose a novel spam filtering framework comprising two components: a rule-based spam filter handling the non-content portion of the emails and a content-based binary classifier spam filter. This framework addresses three drawbacks of existing solutions:

(1) Lack of online real-time spam detection solutions.
(2) Absence of hybrid DL models for content-based binary classifiers.
(3) Absence of hybrid methods for both content-based and non-content-based spam detection.

Using this framework, we mainly focus on a dynamic and multi-layer classification scheme. Our solution consists of a deep-learning-based hybrid model combining CNN and LSTM. The functional testing results for our content-based binary spam classifier show that it outperforms existing approaches by achieving the highest accuracy rate while maintaining a low execution time. We now need to compare it, with respect to several performance metrics, to DELM, a recently presented approach that uses double-hidden-layers extreme ML for data stream classification. DELM can handle various types of concept drift in data stream classification, which is an issue we will address in the immediate future with the expectation of achieving further improvements to our solution. Taking into account the semantics of specific domains, which we did not present in this article, should also bring further improvements for domain-based solutions proceeding from our framework. Thus, in conclusion, we believe the framework we propose constitutes a good starting point for the development of a comprehensive anti-spam software solution. That is, one that can tackle multiple types of spam that can attack an email system from a multitude of angles.

REFERENCES

[1] N. Abroyan. 2017. Convolutional and recurrent neural networks for real-time data classification. In *Proceedings of the 7th International Conference on Innovative Computing Technology (INTECH'17)*. 23–27. DOI: https://doi.org/10.1109/INTECH.2017.8102422

[2] N. Abroyan and R. G. Hakobyan. 2016. A review of the usage of machine learning in real-time systems. *Proceedings of NPUA Information Technologies Electronics Radio Engineering* 19, 1 (2016), 46–54.

[3] A. Alghoul, S. A. Ajrami, G. A. Jarousha, G. Harb, and S. S. Abu-Naser. 2018. Email classification using artificial neural network. *Int. J. Acad. Eng. Res.* 2, 11 (2018).

[4] A. Almomani, S. Atawneh, A. Meulenberg, and E. Almomani. 2013. A survey of phishing email filtering techniques. *IEEE Commun. Surv. Tutor.* 15 (2013), 2070–2090.

[5] A. C. Atluri and V. Tran. 2017. Botnets threat analysis and detection. *Inf. Secur. Pract.* 123 (2017), 7–28. DOI: https://doi.org/org/10.1007/978-3-319-48947-6_2

[6] A. Barushka and P. Hajek. 2018. Spam filtering using integrated distribution-based balancing approach and regularized deep neural networks. *Appl. Intell.* 48, 10 (2018), 3538–3556.

[7] A. Barushka and P. Hajek. 2019. Review spam detection using word embeddings and deep neural networks. In *Proceedings of the IFIP International Conference on Artificial Intelligence Applications and Innovations (AIAI'19)*. 340–350.

[8] N. Bhargava, G. Sharma, R. Bhargava, and M. Mathuria. 2013. Decision tree analysis on j48 algorithm for data mining. *Int. J. Adv. Res.Comput. Sci. Softw. Eng.* 3, 6 (2013).

[9] A. Bhowmick and S. M. Hazarika. E-mail spam filtering: a review of techniques and trends. In *Lecture Notes in Electrical Engineering*, Vol. 443. pp. 583–590. DOI: https://doi.org/10.1007/978-981-10-4765-7_61

[10] J. Carson. 2018. Symantec Internet Security Threat Report 2018: The Top Takeaways. Retrieved from https://thycotic.com/company/blog/2018/04/17/symantec-internet-security-threat-report-2018/.

[11] K. Chen, X. Zou, X. Chen, and H. Wang. 2019. An automated online spam detector based on deep cascade forest). In *Proceedings of the International Conference on Science of Cyber Security (SciSec'19)*. 33–46.

[12] V. Cihan and M. T. A. Hezha. 2019. Comparison of string matching algorithms on spam email detection. In *Proceedings of the International Congress on Big Data, Deep Learning and Fighting Cyber Terrorism (IBIGDELFT'18)*. DOI: https://doi.org/10.1109/IBIGDELFT.2018.8625317

[13] E. G. Dada, J. S. Bassi, H. Chiroma, S. M. Abdulhamid, A. O. Adetunmbi, and O. E. Ajibuwa. 2019. Machine learning for email spam filtering: review, approaches and open research problems. *Elsevier* 5, 6 (2019). DOI: https://doi.org/10.1016/j.heliyon.2019.e01802

[14] Merriam-Webster Dictionary. 2018. Merriam-Webster Dictionary, "Spam - Definition Spam Use by Merriam-Webster." Retrieved from https://thycotic.com/company/blog/2018/04/17/symantec-internet-security-threat-report-2018/.

[15] J. Fernandez-Conde, P. Cuenca-Jimenez, and R. Toldedo-Moreo. 2019. Improving scheduling performance of a real-time system by incorporation of an artificial intelligence planner. In *Proceedings of the International Work-Conference on the Interplay Between Natural and Artificial Computation (IWINAC'19)*. 127–136.

[16] H. A. Ghada and B. M. Rabiei. 2019. Comparison of four email classification algorithms using WEKA. *Int. J. Comput. Sci. Inf. Secur.* 17, 2 (2019).

[17] M. Granik and V. Mesyura. 2017. Fake news detection using naive Bayes classifier. In *Proceedings of the IEEE 1st Ukraine Conference on Electrical and Computer Engineering (UKRCON'17)*. DOI: https://doi.org/10.1109/UKRCON.2017.8100379

[18] R. Team. 2015. *Email Statistics Report.* The Radicati Group, Inc. Palo Alto, CA, USA, Mar. 2015.

[19] G. Jain, M. Sharma, and B. Agarwal. 2019. Optimizing semantic LSTM for spam detection. *Int. J. Inf. Technol.* 11 (2019), 239–250.

[20] K. Jiang, S. Feng, Q. Song, R. A. Calix, M. Gupta, and G. R. Bernard. 2018. Identifying tweets of personal health experience through word embedding and LSTM neural network. *BMC Bioinf.* 19, 210 (2018). DOI: https://doi.org/10.1186/s12859-018-2198-y

[21] S. Kadam, A. Gala, P. Gehlot, A. Kurup, and K. Ghag. 2018. Word embedding based multinomial naive Bayes algorithm for spam filtering. In *Proceedings of the 4th International Conference on Computing Communication Control and Automation (ICCUBEA'18)*. DOI: https://doi.org/10.1109/ICCUBEA.2018.8697601

[22] Z. Khan and U. Qamar. 2016. Text mining approach to detect spam in emails. In *Proceedings of the International Conference on Innovations in Intelligent Systems and Computing Technologies.*

[23] R. Kiran and I. AtmosukartoS. 2005. *Spam or Not Spam—That Is the Question.* Technical Report. University of Washington.

[24] A. Kumar and J. P. Singh. 2018. Location reference identification from tweets during emergencies: A deep learning approach. *Int. J. Disast. Risk Reduct.* 33 (2018), 365–375. DOI: https://doi.org/10.1016/j.ijdrr.2018.10.021

[25] D. Kwon, K. Natarajan, S. C. Suh, H. Kim, and J. Kim. 2018. An empirical study on network anomaly detection using convolutional neural networks. In *Proceedings of the IEEE 38th International Conference on Distributed Computing Systems (ICDCS'18)*. DOI: https://doi.org/10.1109/ICDCS.2018.00178

[26] J. Lu, A. Liu, F. Dong, F. Gu, J. Gama, and G. Zhang. 2019. Learning under concept drift: A review. *IEEE Trans. Knowl. Data Eng.* 31, 12 (2019), 2346–2363. DOI: https://doi.org/10.1109/TKDE.2018.2876857

[27] M. Popovac, M. Karanovic, S. Sladojevic, M. Arsenovic, and A. Anderla. 2018. Convolutional neural network based SMS spam detection. In *Proceedings of the 26th Telecommunications Forum (TELFOR'18)*. DOI: https://doi.org/10.1109/TELFOR.2018.8611916

[28] J. Ramos. 2003. Using TF-IDF to determine word relevance in document queries. In *Proceedings of the 1st Instructional Conference on Machine Learning.* 133–142.

[29] R. M. Ravindran and A. S. Thanamani. 2015. K-means document clustering using vector space model. *Bonfring Int. J. Data Mining* 5, 2 (2015), 10–14.

[30] R. Ruskanda and F. Zakhralativa. 2019. Study on the effect of pre-processing methods for spam email detection. *Indon. J. Comput.* 1 (2019), 109–118.

[31] N. Saidani, K. Adi, and M. S. Allili. 2017. A supervised approach for spam detection using text-based semantic representation. In *Proceedings of the International Conference on E-Technologies (MCETECH'17)*, Vol. 289. Springer, Cham, 136–148.

[32] G. Schryen. 2007. 2007 anti-spam measures. In *2Anti-Spam Measures.* Springer, Berlin, 178. DOI: https://doi.org/10.1007/978-3-540-71750-8_4%7D

[33] P. Sethi, V. Bhandari, and B. Kohli. 2017. SMS spam detection and comparison of various machine learning algorithms. In *Proceedings of the International Conference on Computing and Communication Technologies for Smart Nation (IC3TSN'17).*

[34] M. Shuaib, O. Osho, I. Ismaila, and J. K. Alhassan. 2018. Comparative analysis of classification algorithms for email spam detection. *Int. J. Comput. Netw. Inf. Secur.* 10, 1 (2018), 60–67.

[35] R. Varghese and K. A. Dhanya. 2017. Efficient feature set for spam Email filtering. In *Proceedings of the IEEE 7th International Advance Computing Conference (IACC'17).* 732–737.

[36] L. Vrizlynn. 2017. IEEE 802.11 network anomaly detection and attack classification: a deep learning approach. In *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC'17)*. DOI: https://doi.org/10.1109/WCNC.2017.7925567

[37] Y. Wang, F. Yu, and Y. Wei. 2018. Research of email classification based on deep neural network. In *Proceedings of the 2nd International Conference of Sensor Network and Computer Engineering (ICSNCE'18)*. DOI: https://doi.org/10.2991/icsnce-18.2018.16

[38] H. Yang, Q. Liu, S. Zhou, and Y. Luo. 2019. A spam filtering method based on multi-modal fusion. *Appl. Sci.* 9, 6 (2019). DOI: https://doi.org/10.3390/app9061152

[39] B. Zhang, G. Zhao, Y. Feng, X. Zhang, W. Jiang, J. Dai, and J. Gao. 2016. Behavior analysis based SMS spammer detection in mobile communication networks. In *Proceedings of the IEEE 1st International Conference on Data Science in Cyberspace (DSC'16)*. 2–7.

[40] I. Zliobaite, M. Pechenizkiy, and J. Gama. 2016. An overview of concept drift applications. *Big Data Anal.: New Algor. New Soc.* 16 (2016), 91–114.

Received June 2020; revised August 2020; accepted September 2021